

PARALELNI SISTEMI – nova akreditacija

1. a) Ubrzanje 4 procesorskog sistema je veće od 4 i efikasnost je veća od 1. Količina izračunavanja je ista kao kod sekvencijalne verzije programa. Šta je razlog ovog super-linearnog ubrzanja?
b) Trebate da odlučite koji od dva programa da koristite. Program će se izvršavati na 512 procesorskom sistemu, ali trenutno na raspolaganju imate svega 16 procesora za merenje performansi. Tokom merenja prikupljene su informacije navedene u tabelama, gde je S ubrzanje, e eksperimentalno utvrđena serijska frakcija, i p broj procesora. Koji program će dati bolje performanse na 512 procesorskom sistemu. Dati kratko objašnjenje.

Program A

p	2	4	8	16
S	1.88	3.12	4.5	5.8
e	0.06	0.09	0.11	0.12

Program B

p	2	4	8	16
S	1.79	2.9	4.3	5.7
e	0.15	0.15	0.15	0.15

2. Objasniti GCD test za detekciju loop-carry zavisnosti. Pomoću GCD testa utvrditi da li u sledećem kodu postoji loop-carry zavisnost
- ```
for p = 1, P
for q = 1, Q
for r = 1, R
for s = 1, S
A(2p + 2q - r + 3s) = A(p - 6q + 5r + 3) * B(r + s - 4q) + A(p - r + 2s + q + 2)
```
3. Dati primer sprežne mreže kod koje stepen čvora raste logaritamski sa porastom broja čvorova. Skicirati mrežu sa 16 čvorova.
4. Pretpostavimo da imamo multiprocesorski sistem sa 256 procesora. Svaki procesor ima lokalnu keš memoriju kapaciteta 1 MB. Kapacitet glavne memorije je 1GB. Veličina keš bloka je 64 B.
- Ako se koristi write-once protokol za postizanje keš koherencije koliko ukupno bitova stanja zahteva implementacija ovog protokola.
  - Ako se koristi direktorijumska šema sa potpuno preslikanim adresarima koliko ukupno bitova stanja je potrebno za implementaciju protokola.
  - Koju od ove dve šeme biste iskoristili za postizanje keš koherencije? Obrazložiti odgovor.

5. a) Pomoću OpenMP paralelizovati sledeći kod

```
C[0] = 1;
for (i=1; i<N; i++){
C[i] = C[i-1];
for (j=0; j<N; j++){
C[i] *= A[i, j] + B[i, j]; }}
```

- b) Sledeći OpenMP program se izvršava na 4-jezgarnom procesoru. Koliko se ubrzanje može postići za veliko  $N$ ? Vreme potrebno za testiranje u if naredbi zanemariti. Usvojiti da ostale operacije traju 1 vremensku jedinicu.

```
float total = 0.0;
#pragma omp parallel for
for (int i=0; i<N; i++) {
if (i % 16 < 8) // usvojiti da testiranje zahteva 0 operacija
out[i] = 1 * in[i]; // 1 op
else
out[i] = 2 + in[i]; // 1 op
}
for (int i=0; i<N; i++)
total += out[i]; // 1 op
```

- a. Napisati MPI program koji realizuje množenje matrice  $A_{n \times n}$  i matrice  $B_{n \times n}$ , čime se dobija rezultujuća matrica  $C_{n \times n}$ . Matrice A i B se inicijalizuju u master procesu. Broj procesa je p i uređeni su kao matrica  $q \times q$  ( $q^2 = p, q = n$ ). Master proces distribuira odgovarajuće vrste matrice A i odgovarajuće kolone matrice B svim procesima. Nakon distribuiranja, procesi u i-toj ( $i=0, \dots, q-1$ ) vrsti matrice procesa sadrže i-tu vrstu matrice A. Nakon distribuiranja, procesi u j-toj koloni matrice procesa sadrže j-tu ( $j=0, \dots, q-1$ ) kolonu matrice B. Nakon toga, svaki proces obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata koji se prikazuje u master procesu. Predvideti da se slanje delova matrica svakom procesu za svaku matricu obavlja odjednom, direktno iz matrice i to korišćenjem Point-to-Point operacija. Za ostatak izračunavanja koristiti grupne operacije. Nije dozvoljeno kreiranje novih komunikatora.
- b. Napisati MPI program koji će omogućiti komunikaciju samo procesa u donjoj trougaonoj matrici (bez glavne dijagonale) u kvadratnoj mreži procesa kreiranjem komunikatora. Obezbediti da svaki proces ove grupe dobije po jednak broj elemenata niza struktura (matični broj, ime i prezime i prosečna plata) gde svaka struktura predstavlja podatke o zaposlenom.

7. Korišćenjem CUDA tehnologije na programskom jeziku C/C++ napisati program koji detektuje i prebrojava otkucaje srca iz niza podataka koji predstavljaju EKG signal. Signal je niz realnih brojeva koji predstavljaju napon u jedinici vremena pročitani sa EKG uređaja. Otkucaj srca je predstavljen lokalnim maksimumom, čija je vrednost veća od susednih vrednosti sa leve i desne strane i premašuje unapred definisan prag  $\alpha$ , kako bi se izbegli šumovi. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za nizove proizvoljne veličine.

8. Data je petlja koja predstavlja Floyd Voršalov algoritam analize grafova za nalaženje najkraćih putanja u težinskom usmerenom grafu (path je kvadratna matrica dimenzija  $N \times N$ ):

```
for (int k = 0; k < N; k++) {
 for (int i = 0; i < N; i++) {
 for (int j = 0; j < N; j++) {
 if (path[i][j] > path[i][k] + path[k][j])
 path[i][j] = path[i][k] + path[k][j];
 }
 }
}
```

a) Da li je moguće izvršiti paralelizaciju date petlje bez modifikacija? Obrazložiti. Ukoliko postoje zavisnosti, navesti koje su i objasniti ih.

b) Transformisati petlju tako da paralelizacija bude moguća i paralelizaciju izvršiti korišćenjem OpenMP. Objasniti implementirano rešenje.

c) Razvijanjem petlje za konkretne vrednosti N i elemente matrice path testirati sekvencijalno i paralelno rešenje i pokazati da je petlja korektno paralelizovana rešenjem pod b).

**Napomena:** zadaci koji daju odgovor samo na stavku b) neće biti bodovani.